



Thanks for taking the time to come out tonight.

I'm Big Joe from Zengenti.

We are a small company in Shropshire. About 70 nerds.

We do websites for universities and local authorities.

I actually don't do any websites, I work in the hosting team.

We maintain a private cloud to run the websites.

We use a combination of Ansible and Python  
maintaining about 3000 servers.

But, tonight Matthew,

I am going to talk about, **why all code sucks**.

We all know good code,  
or at least we think we do.

But I should probably define what I mean by sucky code.

*"Programs are meant to be read by humans and only incidentally for computers to execute."*  
— Donald Knuth

└─What is sucky code?

My short answer is . . . **sucky code is hard to read.**

No need take my word for it.

Donald Knuth, the Yoda of Computer Science,  
says that code is for humans to read  
and sometimes for computers to run.  
He is all about the readability.

2025-04-16

# All Code Sucks

## └ The Great Hunt for Non Sucky Code

For about about 25 years now,  
I have been looking for code that doesn't suck.  
And trying to produce code that doesn't suck.  
I've worked in companies large and small.  
But pretty much, all the code sucked.

This does beg the question, why does all code suck?

The Great Hunt for Non Sucky Code



## └ Why Code Sucks

On the whole I think the odds are against us.  
Straight out of the gate,  
half of everything is going to be below average.  
Well, below the median.  
That's just the way averages work.

## └ Why Code Sucks

- Half of everything is below average
- Sturgeon's Law

Then there is Sturgeon's Law.

Sturgeon was explaining why most science fiction is low quality.

And came up with the pithy answer,

“90% of everything is shit”.

The observation works here too.

## └ Why Code Sucks

- Half of everything is below average
- Sturgeon's Law
- The 3 Year Old Programmer

Then there is our experience.

When I started out

the average programmers experience was 3 years.

Now 25 years later that hasn't changed.

Probably because demand outstrips supply.

Which has been nice.

So if the odds are against us

maybe the organizations we work for, will help.

Or perhaps not.

## └ Why Code Sucks

The romantic image of a software startup is a couple of guys in a garage.

I have actually see this quite a bit.

For most start ups the two guys are the dad and the son.

The dad is the salesman.

And the son is the programmer.

They are inherently under resourced.

Writing readable code is a luxury they can't afford.

- Software Startups
- Summer Student Projects

## └ Why Code Sucks

The other kind of startup I have seen occurs in big companies.

The summer student project.

Alternatively called the **unsupervised use of new technology**.

All the experienced programmers are on holiday or busy.

So they give the new technology to the summer students, who give it a go.

And if it runs they put it into production.



## └ Why Code Sucks

- Software Startups
- Summer Student Projects
- Prototypes in Production

This last point is also a general point.  
Any software that appears to work goes into production.  
Not because anyone thinks it's a good idea  
but because there is a commercial imperative.  
Having learnt from the prototype,  
the plan was to throw it away and build it properly.  
But that never happens.  
It is always put into production.

Just in case you haven't lost all hope  
I would say that suck  
is actually built into human psychology.

## └ Why Code Sucks

We are unaware of our ignorance.

Because we are ignorant of it.

We don't know what we don't know.

But writing code is about explaining things in detail.

So it is no surprise that we struggle.

- The Illusion of Explanatory Depth
- Availability Bias

## └ Why Code Sucks

A couple of years ago we used a static analysis tool on our code.  
A gadget called 'CodeScene' for anyone that is interested.  
To my surprise it said most of our code was good.  
But it did point to the five worst files.  
Those are the files I spend most of my time working on.  
So those are the bits I remember.  
It turns out all code doesn't actually suck,  
I just feels like it.

Nevertheless, if that is what you are working on,  
makes sense to tool up for sucky code.

## └─What To Do

- **Discovery** Log behavior
- **Exploration** Scripted investigation
- **Automating** Test harnesses (or clamps)
- **Deconstruction** Separate the target code
- **Enable** Switching with a feature flag
- **New Code** Side by side rewrite

This is my adapted 6 step plan to work on sucky code.  
An initial discovery phase is to find out how the code is actually used.  
Then we can explore the code and find it's quirks.  
Our intermediate goal is to write some automated tests.  
To protect us from making changes that might break things.  
With the tests in place we can start to deconstruct the code.  
And target the bits that need attention.  
Using a feature flag will enable us to switch between  
old and new code.  
Finally we can get on with the job of writing new code.

I have skipped the step about  
getting control of the development environment.  
We use ephemeral environments at Zengenti to keep things clean.  
But that is a whole other talk, for another day.

## └─What To Do

- **Discovery** Log behavior
- **Exploration** Scripted investigation
- **Automating** Test harnesses (or clamps)
- **Deconstruction** Separate the target code
- **Enable** Switching with a feature flag
- **New Code** Side by side rewrite

As a demo I have an API with one end point.  
You can enjoy the squiggles, however, it does run.

**make app**

Let's take a look and see what it does.

**Open <http://127.0.0.1:8000/docs>**

**Try 2 and 42**

I can't really tell what is going on.

We really need to discover how the end point is being used.

So adding some logging is a good idea.

**demo01log**

Michael Feathers says don't change any thing until you have tests,  
but he admits that sometimes you have to add stuff like logging.

**Try 2, 4 and 42**

We have a log of actual user behavior.

## └─What To Do

- **Discovery** Log behavior
- **Exploration** Scripted investigation
- **Automating** Test harnesses (or clamps)
- **Deconstruction** Separate the target code
- **Enable** Switching with a feature flag
- **New Code** Side by side rewrite

Now we know how the end point is used.

We can start to explore the code a bit more.

### **Open tests/app.http**

We can use http scripts to mimic the UI behavior.

### **demo2http01**

There's a JetBrains client for this or in this case a VSCode extension.

But you could equally well use a Jupyter notebook.

### **demo2http02**

This allows us to do a more systematic investigation.

Looks like a string is coming back.

Not a number.

### **Fix inverted commas**

We can get a feel for the behavior and quirks.

We might even start to get an insight into the author's intent.

## └─What To Do

- **Discovery** Log behavior
- **Exploration** Scripted investigation
- **Automating** Test harnesses (or clamps)
- **Deconstruction** Separate the target code
- **Enable** Switching with a feature flag
- **New Code** Side by side rewrite

With this knowledge we can write some proper automated tests.  
The kind of tests we could use in a CI/CD pipeline.

**Open tests/app.test.py**

These are the clamps that will hold the code in place.  
As Michael Feathers calls them.

We run these tests every time we make a change.  
To make sure we haven't broken anything.

**demo3test01****demo3test02**

This will give us the confidence to make changes.  
Every time we make a change we run the tests.

**make test**

We get a little report each time.  
To let is know it is alright to continue.

## └─What To Do

- **Discovery** Log behavior
- **Exploration** Scripted investigation
- **Automating** Test harnesses (or clamps)
- **Deconstruction** Separate the target code
- **Enable** Switching with a feature flag
- **New Code** Side by side rewrite

With the automated tests in place.

We are ready to start deconstructing the code.

**Open app.py**

We can start to separate the target code.

**demo4function01**

Since we don't really know what the code does.

We can't give it a meaningful name.

Michael Feathers suggests just mashing the keyboard.

We can check that it works in all it's ugliness.

By rerunning the tests.

**make test**



## └─What To Do

- **Discovery** Log behavior
- **Exploration** Scripted investigation
- **Automating** Test harnesses (or clamps)
- **Deconstruction** Separate the target code
- **Enable** Switching with a feature flag
- **New Code** Side by side rewrite

Stay with me we are getting close.

Now we can enable a switch to control any new code.

There are lots of tools to do this.

We favour Unleash because it comes for free with Gitlab.

**Open /joejcollins/alan-tracy/-/feature\_flags**

Here I have a feature flag to turn a function on and off.

I can use this to switch between the old and the new code.

**demo5flag01**

**make test**

So lets rewrite the code.

**demo6new01**

This is actually a cut and paste from ChatGPT.

It is still a bit of a mess, but I am running out of time.

Things still work.

The code still sucks, but it sucks less.

## └─ The Challenge

*"It's not that hard..."*

— Billy Beane

*"It's incredibly hard"*

— Ron Washington

To quote Moneyball,  
this challenge is both easy and difficult.  
The example was one function and one file.  
But the code we are working with has 3500 files.  
Some is hard to read,  
some has been improved and some it half way between.  
But it often feels like we have three different code bases.  
All jumbled up.

It turns out the improving code is easy  
but managing the process is hard.  
Knowing which bits to improve really hard.  
I have no answer to that, I am kind of hoping you do.

**Thank You!**

Joe J Collins



Zengenti

Thank you for listening.

I'm Big Joe from Zengenti.

If you thought this was interesting,  
and would like to work with us,  
please get in touch.

If you have other ideas how to work with sucky code,  
I'm all ears.

Or if you have an idea how to manage the process,  
Let's talk.